Conditions for Direct Behavior Specification in Multi-Agent Reinforcement Learning

Yousuf Rashid
University of Waterloo
Waterloo, Ontario, Canada
y2rashid@uwaterloo.ca

Abstract

Direct behavior specification via constrained reinforcement learning replaces hand-tuned reward weights with target rates that Lagrange multipliers enforce on their own. The method was developed and evaluated with off-policy SAC on single-agent navigation tasks. We test it with ten agents sharing one policy, on-policy PPO, a scripted opponent that reacts within each episode, and a continuous physics simulation fit to real match tracking data. A movement constraint transfers: across five targets, a frozen-policy probe puts the achieved rate within three points of the requested one on average. A crowding constraint does nothing at any threshold we swept, because it is an upper bound the policy already satisfies at initialization. The method's threshold semantics assume a binding constraint, and this is what a non-binding one looks like in practice. We give a one-pass check that catches it, and argue that a rate the designer wants hit needs a bound on both sides. Counting the movement indicator over the team instead of per agent breaks tracking in all five seeds. A preregistered follow-up that swaps the multiplier update for a PID controller restores tracking in two of three seeds on training logs and one of three on the probe, and breaks tracking in the per-agent control, so the failure comes from the update rule and the indicator together rather than from the counting unit alone. We also report a budget-matched baseline that removes a 13 to 17 point success cost we had first attributed to stacking constraints.

1 Introduction

Reward engineering is how most applied reinforcement learning projects encode what an agent should and should not do. A practitioner writes a weighted sum of terms, trains, watches the resulting behavior, adjusts the weights, and repeats. The weights carry no interpretable meaning, the search over them is expensive, and the agent will exploit any term that admits a shortcut.

Roy et al. [8] propose an alternative. Rather than assigning a weight to a behavior, the practitioner states a target rate for it, and a Lagrange multiplier finds the weight that achieves that rate. Their framework restricts cost functions to indicators, so each threshold reads as a probability the designer can reason about directly, and it handles several constraints at once through multiplier normalisation and a bootstrap constraint on task success. They evaluate it with off-policy SAC on single-agent navigation tasks carrying up to five simultaneous behavioral constraints.

The regime matters because it is where the method would actually be used. The motivation for direct behavior specification is a designer who wants an agent to behave a particular way and does not want to discover the weights by trial and error. In games that agent is rarely alone. It is one of a squad, a patrol or a team, sharing a policy for reasons of memory and training cost, acting against an opponent that is itself reacting, in a simulation with real physics rather than a grid. Each of those properties changes what a constraint can do. A shared policy means one specification governs ten different roles. An opponent that reacts means the behavior rate the constraint measures moves for reasons the agent does not control. On-policy training means the multiplier and the policy update from the same samples. None of this is exotic, and all of it is outside what the method has been shown to handle.

We test the framework in that regime. Our environment places ten attacking agents under a single shared policy against a scripted defending side that reacts to the attack within each episode, in a continuous physics simulation calibrated against real match tracking data, trained on-policy with PPO. We report four findings.

- The framework transfers for an upper-bound constraint the policy violates. Across five targets at three seeds each, a frozen-policy probe gives a slope of 0.93 between requested and achieved rate, a mean absolute error of 3.0 points on target means and 4.4 points per run. The probe does not separate the two lowest targets (Section 6.1).
- A crowding constraint has no effect at any threshold we swept, because it is an upper bound the policy satisfies from initialization. Roy et al. state that their threshold semantics assume a binding constraint. We show what the failure looks like, give the check that catches it before training, and argue that a target rate needs a bound on both sides (Section 6.2).
- Counting the movement indicator over the team rather than per agent breaks tracking in all five seeds. A preregistered 2×2 that swaps the multiplier update rule for a PID controller says the update rule is responsible under our preregistered measure, and is mixed under the probe. The same PID controller breaks tracking in the per-agent control, so neither the update rule nor the counting unit explains the failure on its own (Section 6.4).
- A budget-matched baseline removes a 13 to 17 point success cost we had first attributed to constraint stacking. We report the confound because the comparison that produced it is a natural one to draw (Section 6.3).

2 Related Work

Constrained MDPs and Lagrangian methods. Constrained MDPs [2] extend the MDP to a policy that maximizes return subject to bounds on auxiliary cost functions. Two families of solution method dominate. Lagrangian methods fold the constraints into a single objective and search for a saddle point, as in reward-constrained policy optimization [12], while projection methods take an unconstrained step and map the policy back into the feasible set, as in constrained

policy optimization [1]. Both were developed for safety, where the constraint must hold throughout exploration and the quantity of interest is cumulative violation during training.

The Lagrangian family has a known failure mode that is directly relevant to us. The multiplier update is integral control on the constraint error, so it accumulates while a constraint is violated and overshoots once the policy crosses the threshold. Stooke et al. [11] add proportional and derivative terms to damp this. Our baseline updates the multiplier logits with Adam, and Section 6.4 compares it directly against a PID update in the style of Stooke et al.

Direct behavior specification. Roy et al. [8] turn the constrained MDP into an interface for designers rather than a safety mechanism. Their three components are the ones we restate in Section 4: cost functions restricted to indicators, so each threshold is a probability between zero and one; multipliers normalized on a simplex, so no constraint can dominate the update without bound; and a bootstrap constraint on task success, which lends its multiplier to the main objective while the optimizer is still searching for a feasible policy.

Their evaluation is single-agent navigation in two environments, an arena with lava, energy and look-at-marker constraints and a larger open world, with up to five behavioral constraints active at once. The learner is SAC, replicated with TD3. The hazards do not move and the agent has no opponent. Our contribution is not to add constraints beyond their five. It is to ask whether the interface still behaves as an interface when the agent is one of ten sharing a policy, when training is on-policy, and when the environment pushes back.

Safe multi-agent RL.. Constrained reinforcement learning has been extended to multiple agents. Gu et al. [4] formulate safe MARL as a constrained Markov game and give two algorithms, MACPO and MAPPO-Lagrangian, with monotonic improvement and constraint satisfaction guarantees per iteration, evaluated on safe multi-agent MuJoCo and robosuite benchmarks. Earlier, CMIX [7] handled peak cost constraints by modifying the reward under a QMIX-style mixer.

That line and ours differ in what the constraint is for. Those constraints encode safety, they are chosen by the benchmark rather than by a designer, and the threshold is a budget to stay under. A designer using direct behavior specification names a behavior and a frequency, and the method is worth using only if the agent then exhibits that behavior at that frequency. Undershooting a safety budget is a success. Undershooting a specification is a miss, and the standard machinery treats both the same way, because both are written as upper bounds. Our crowding result in Section 6.2 is a case that safe MARL would score as fully satisfied and that a designer would score as broken.

Emergent side effects. Constraining one behavior can produce another that nobody specified, which is the negative side effects problem of Amodei et al. [3]. Two lines of work respond to it by penalizing an agent for changing the world more than the task requires, either by measuring reachability of states the agent could otherwise have reached [5] or by measuring loss of the agent's own attainable utility across auxiliary reward functions [13]. Those methods define the penalty in advance and do not require the designer to name the side effect. The workflow we study is the opposite one: observe the side effect after the fact, name it, and add a constraint on it. Section 6.2 is about why that second step can fail without saying so.

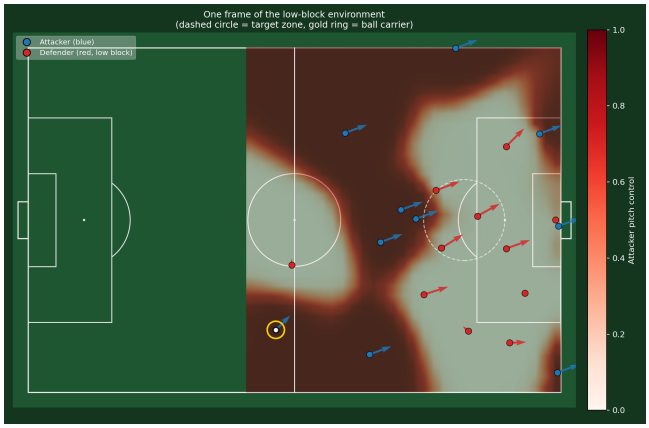


Figure 1: A single tick of the environment. Ten attackers (one holding the ball) face eleven defenders in a 5-4-1 low block. The shaded region is the attacking side's pitch control, computed on a 31×34 grid of 2 m cells. The circle at $(86, 34)$ is the target zone; success requires the carrier inside it with mean attacking control over its cells of at least 0.45.

Reinforcement learning in football. Football is a standing multi-agent benchmark. Google Research Football [6] provides a full eleven-a-side simulator with either one agent controlling the team or one per player, and is where most multi-agent RL work on the sport is run. Separately, TacticAI [14] shows that models built on real tracking data are accepted by practitioners, and it uses the same pitch control formulation we build our reward on.

We use a custom environment rather than Google Research Football for one reason. Our experiments need behavioral indicators that are cheap to measure per agent per tick, thresholds that can be swept, and a physics model whose parameters are fitted rather than fixed, and we need to be able to disable one constraint and rerun. A general-purpose game simulator makes each of those harder than writing the scenario directly.

3 Environment and Task

3.1 The low-block scenario

Ten attackers share a single policy against eleven defenders holding a 5-4-1 low block on a 105×68 m pitch. The attacking side starts in its own half with possession and must work the ball into a target zone, a disc of radius 8 m centred at $(86, 34)$, in front of the defending goal.

An episode succeeds when the ball carrier is inside the zone and the attacking side holds mean pitch control of at least 0.45 over the zone's cells. Both conditions matter. Reaching the zone with the ball is not enough if the defense has collapsed around it, which is what makes the task about creating space rather than about navigation. An episode fails on a turnover, either a lost ground duel, an intercepted pass, or an offside, and times out after 200 ticks at 0.1 s per tick, or 20 seconds of simulated play.

The reward is sparse at termination, +1.0 for success, −0.3 for a turnover and −0.5 for a timeout, with dense potential-based shaping on the ball's distance to the zone centre. The potential is $\phi = -0.03\,\|p_{\text{ball}} - c_{\text{zone}}\|$ and is zeroed at termination so the shaping telescopes and leaves the optimal policy unchanged.

The low block is the configuration worth studying because it removes the easy solution. Against a high line the attackers can play direct and beat the defense with pace. Against a deep block there is no space behind, so the task can only be solved by moving defenders, which requires the ten agents to do different things at the same time. A shared policy has to produce that differentiation from identical weights, and the behavioral constraints we impose act on exactly the movement patterns that produce it.

The defending side is not learned. Its block depth tracks the attackers' four deepest players and the ball's x position, and its lateral centroid follows the ball with fitted gains, so the defense responds to the attack within an episode without changing across training. This keeps the opponent fixed as an experimental control while still presenting a moving target within an episode.

3.2 Physics and calibration

Players integrate at 0.1 s under a maximum speed of 5 m/s and a maximum acceleration of 7 m/s^2. Space is scored with the pitch control model of Spearman et al. [9, 10], which integrates for every player at every grid point the probability that the player arrives and controls the ball first. We evaluate it on a 31×34 grid of 2 m cells covering the attacking 62×68 m of the pitch.

Four components are fitted to tracking data rather than chosen by hand, each with a held-out check reported in the corresponding script. Carrying speed is 0.81 of free-running speed, taken as the median per-player ratio of 95th percentile peak speed on carrying runs to peak speed on ball-free runs, over 124 carrying runs and 2957 free runs from twelve players, restricted to efforts above 4 m/s. Only the ratio transfers, since real players top out near 9 m/s against our cap of 5. Pass speed scales with pass length, fitted to 793 passes and binned by length, replacing the constant 15 m/s the engine used before. Defender formation parameters are fitted per slot and per frame from labelled freeze frames, with a resampling procedure that refits on a subsample and evaluates on frames the fit never saw. Starting formations for both sides are drawn from a correlated Gaussian fitted the same way. The control threshold that defines success, 0.45, is the 25th percentile of the control level real shots were taken from, so it admits the situations three quarters of real shots occurred in.

Two of these fits are worth reporting as negatives. The first anchor we tried for the defender calibration, a target possession retention of 0.90, has no solution in this model at any threshold, and the dribble-speed calibration failed against an absolute speed anchor before the ratio anchor worked. Both are documented in the calibration scripts.

3.3 Observation and action spaces

Each attacker observes a 120-dimensional vector: its own position and velocity, relative position, relative velocity and carrier flag for each of nine teammates, relative position and velocity for each of eleven defenders in fixed roster order, a seven-element ball block, its distance to the zone and the current zone control, an offside block, the normalized clock, four reserved constraint-rate slots, and a one-hot agent index. The one-hot is what lets ten agents under identical weights learn distinct roles.

Each attacker emits three discrete choices per tick: one of nine directions, eight compass headings plus stationary; one of three speed settings at 0.3, 0.6 and 1.0 of maximum; and one of ten ball actions, hold or a pass to one of nine teammates. The joint action is therefore 30 discrete choices per tick, and they are correlated in two ways that matter. The ball dimension is only read for whichever agent currently holds the ball, and the other nine are masked to hold, so nine tenths of that dimension is inert on any given tick and which tenth is live changes with possession. And because the outcome depends on where all ten players are, the value of any one agent's movement choice depends on the other nine, which the shared policy has to represent without communicating.

3.4 Reward and behavioral indicators

We use two behavioral indicators. The first, C_{slow}, fires for an individual attacker on a tick when its speed is below 2 m/s. Since the three speed settings correspond to 1.5, 3.0 and 5.0 m/s, this indicator distinguishes the lowest gear from the other two, modulo acceleration between settings. The second, C_{crowd}, fires when at least four attackers are within the zone radius of the zone centre.

Both behavioral constraints are upper bounds, $\hat{J}_k \le \tilde{d}_k$. The only lower bound in the problem is the bootstrap success constraint of Section 4. This matters for everything that follows. An upper bound can only push a rate down, and only while the rate is above the threshold. The unconstrained policy runs C_{slow} at 0.88 (Table 2), so every slow target we sweep starts out violated.

The two indicators also differ in what they count over, which we return to in Sections 4.6 and 6.4. The slow indicator is computed per attacker, so each of the ten agents contributes its own cost signal on every tick and the batch average in Equation (4) runs over attacker-ticks. The crowding indicator is a team-level event. It is evaluated once per tick over all ten positions and the same scalar is then broadcast to every agent, so the average runs over ticks rather than attacker-ticks and every agent receives an identical cost regardless of whether it was personally inside the disc.

3.5 Simulator throughput

Pitch control is a large amount of small arithmetic evaluated at every grid cell for every player on every tick, across all parallel environments. In our setup it was the training bottleneck rather than the policy update. We reimplemented it as a CUDA kernel in raw C++ through CuPy's `RawKernel` and kept the NumPy implementation as a reference rather than replacing it. The kernel gives 5.4 times the environment steps per second, and the two backends agree to within 9×10^{-4} on every output, checked against a stored fixture on every run rather than verified once. The experiments in this paper total roughly fifty training runs at budgets between 2.5M and 10M steps, which the NumPy backend would not have made practical.

4 Method

We use the framework of Roy et al. [8] without modification to its constraint machinery. This section restates the parts we rely on, citing their equation numbers, and then lists the places our implementation departs from theirs. Equations (1) through (7) are theirs.

4.1 Constrained objective

A constrained MDP [2] seeks a policy of maximal expected return among those satisfying a set of cost bounds. Following [8, Eq. 6],

$$\pi^* = \arg\max_{\pi \in \Pi} J_R(\pi) \quad \text{s.t.} \quad J_{C_k}(\pi) \le d_k, \quad k = 1, \dots, K. \tag{1}$$

4.2 Indicator cost functions

Roy et al. restrict attention to cost functions that are indicators [8, Eq. 9],

$$C_k(s, a) = \mathbb{I}(\text{behavior } k \text{ is met in } (s, a)), \tag{2}$$

and observe that rewriting the discounted sum of such a cost as an expectation over the visitation distribution makes it a rescaled probability [8, Eqs. 10–13],

$$J_{C_k}(\pi) = Z(\gamma, T) \Pr_{(s,a)\sim x_\pi} [\text{behavior } k \text{ met in } (s, a)], \tag{3}$$

with $Z(\gamma, T) = \sum_{t=0}^{T} \gamma^t$. Dividing the constraint through by Z leaves a normalized threshold $\tilde{d}_k \in [0, 1]$ that reads as a desired behavior rate. This is what lets a designer write 0.75 and mean three quarters of ticks. In practice the constraint is estimated as a batch average [8, Eq. 14],

$$\tilde{J}_{C_k}(\pi) := \frac{1}{N} \sum_{i=1}^{N} C_k(s_i, a_i). \tag{4}$$

4.3 Multiplier normalisation

Unnormalized multipliers on a hard-to-satisfy constraint grow without bound and destabilize learning. Roy et al. normalize them with a softmax over base parameters z_k and a dummy variable a_0 [8, Eq. 15],

$$\lambda_k = \frac{\exp(z_k)}{\exp(a_0) + \sum_{k'=1}^{K} \exp(z_{k'})}, \qquad \lambda_0 := 1 - \sum_{k=1}^{K} \lambda_k, \tag{5}$$

which gives the min-max problem [8, Eq. 16],

$$\max_{\pi} \min_{z_{1:K} \ge 0} \lambda_0 J_R(\pi) - \sum_{k=1}^{K} \lambda_k \left(J_{C_k}(\pi) - \tilde{d}_k \right). \tag{6}$$

One consequence of the softmax is easy to miss. The multipliers share a single unit of weight, so a constraint can only gain weight by taking it from another constraint or from the task. Section 6.4 turns on this.

4.4 Bootstrap constraint

Because $\lambda_0 = 1 - \sum_k \lambda_k$, the weight on the main objective is squeezed while the optimizer searches for a feasible policy, leaving little traction on the task. Roy et al. add a lower-bound success constraint indexed $K{+}1$ and lend its multiplier to the main objective [8, Sec. 4.3],

$$\tilde{\lambda}_0 := \max(\lambda_0, \lambda_{K+1}), \tag{7}$$

Table 1: The places our implementation departs from Roy et al. [8]. Equations (1)–(7) are used unmodified.

Deviation	Reason
On-policy PPO in place of off-policy SAC	Their Appendix A notes that the multiplier objective is optimized on-policy even inside an off-policy learner, and that reconciling the two forces a tradeoff between estimate variance and sample efficiency. PPO removes the tension at the cost of sample reuse.
Combination at standardized advantages rather than at Q-values	PPO has no Q function to combine at. Standardizing each advantage channel before combining keeps the terms on a common scale, which is the role their $K{+}2$ separate critics play.
Adam on the logits z_k	A standard choice for the multiplier optimizer. It moves each logit by roughly the learning rate per update whatever the size of the violation, which matters in Section 6.4.

substituted for λ_0 in Equation (6). We set the success floor at 0.40. Once success clears it, λ_{K+1} decays and the task weight passes back to λ_0. We call that moment the handover and log the step at which it happens. In our per-agent control runs it falls between 1.27M and 1.97M steps (Table 5).

4.5 Where our implementation differs

Their algorithm applies Equation (6) at the Q-values of $K{+}2$ separate critics. We apply it at generalized advantages, standardized per channel:

$$\hat{A} = \tilde{\lambda}_0 \bar{A}_R \; + \; \lambda_{K+1} \bar{A}_{K+1} \; - \; \sum_{k=1}^{K} \lambda_k \bar{A}_k, \tag{8}$$

where $\bar{A}$ denotes a batch-standardized advantage. This is the only structural change. The sign convention, the bootstrap term and the multiplier objective follow (6) and (7) directly. We compute cost advantages under a separate discount γ_k and scale their targets by $(1 - \gamma_k)$, which implements the $Z(\gamma, T)$ normalization of (3) for truncated returns.

The logits z_k are updated by Adam at a learning rate of 0.03 on the constraint violation. Adam normalizes its step, so under a persistent violation each logit climbs by about 0.03 per update regardless of whether the violation is small or large. Section 6.4 replaces this update with a PID controller and leaves everything else in this section unchanged.

4.6 Indicator granularity

A shared policy raises a design question that does not arise with a single agent: what unit does the indicator in Equation (2) count over? A slowness constraint can be written per agent, firing for each attacker below 2 m/s, or per team, firing once per tick when the team's mean speed is below 2 m/s. Roy et al. have no occasion to make this choice, since their agent acts alone.

The two versions share a speed threshold but they are not the same specification. Our per-agent policies trained to a slow rate of 0.75 show a team-mean rate of about 0.95 under the probe, and the unconstrained policy shows 0.999. A team-mean target of 0.75 therefore asks for a much faster team than a per-agent target of 0.75,

and it starts from an indicator that is saturated at one. A team-mean target of 0.75 is still reachable: frozen per-agent policies from our sweep span team-mean rates from 0.65 to 0.87 at success between 0.68 and 0.82.

We use per-agent indicators for C_{slow}. Section 6.4 shows what happens under a team-mean indicator at the same threshold.

4.7 Notation

Symbol	Meaning
J_R, J_{C_k}	Expected return; expected discounted cost k
$\tilde{d}_k$	Normalized target rate for constraint k
$\hat{J}_k$	Measured rate of behavior k
λ_k	Normalized multiplier on constraint k
λ_0, $\tilde{\lambda}_0$	Weight on task reward, before and after the bootstrap floor
$K{+}1$	Index of the success (bootstrap) constraint

5 Measurement

Frozen-policy probe against training logs. Averaging an indicator over training logs mixes behavior from every policy the run passed through. We also evaluate each frozen final policy over 150,000 attacker-ticks. The two disagree: on the slow sweep, training logs report a mean absolute tracking error of 2.4 points per run against the probe's 4.4. The log figure flatters the method. We report the probe for behavior rates wherever we have it, give the log figure next to it where both exist, and say which instrument each number comes from. Success rates and multiplier values in Sections 6.3 and 6.4 come from training logs unless marked.

Wilson intervals. The probe runs for a fixed number of ticks rather than a fixed number of episodes, so it cuts off mid-episode at whichever episode is running when the window ends. Short episodes are over-represented in what it captures, and short episodes are disproportionately turnovers. Counting every episode the probe touches therefore biases success downward. We report success only over episodes that terminated inside the window, with Wilson score intervals rather than normal approximations, since success rates near the ends of the range and modest episode counts are exactly where the normal interval misbehaves.

Tail means and what they hide. A run's achieved rate is not a single number. The multiplier and the policy chase each other, so the measured rate oscillates, and reading the final logged point turns that oscillation into noise. We average the last 20% of logged points instead. Some of our runs log every 50 updates and some more often, so the tail of a 66-point log and the tail of a 326-point log cover the same steps at different resolution.

This convention hides something, and it is worth showing where. In `sweep_crowd10_s3` at 2.5M steps the tail mean is 0.044, comfortably under its requested 0.10, so the run reads as feasible. Individual points inside that same window reach 0.112, above the threshold. Whether that run satisfied its constraint depends entirely on which summary is used, and neither summary is wrong. We use the tail mean throughout and note here that a feasibility call in this setting is a statement about a window, not about a policy.

Table 2: Slow-constraint sweep, three seeds per target. Probe columns come from frozen final policies; the log column is the tail mean of training logs. The last row is the unconstrained baseline, one seed at 10M steps.

$\tilde{d}$	Probe $\hat{J}_{\text{slow}}$	Log $\hat{J}_{\text{slow}}$	Speed	Probe success
0.45	0.478 ± 0.105	0.450	2.05	0.760 ± 0.083
0.55	0.496 ± 0.030	0.546	2.02	0.863 ± 0.025
0.65	0.680 ± 0.010	0.659	1.63	0.739 ± 0.076
0.75	0.744 ± 0.036	0.745	1.49	0.741 ± 0.074
0.85	0.819 ± 0.016	0.794	1.33	0.666 ± 0.004
none	0.883	n/a	1.13	0.601

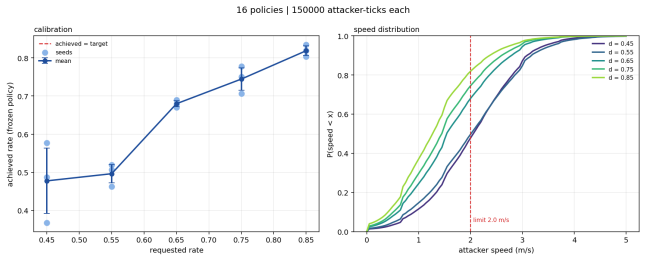


Figure 2: Left: achieved against requested slow rate under the frozen-policy probe, five targets at three seeds. Right: the cumulative distribution of attacker speed for each target, with the 2 m/s indicator threshold dashed. The two lowest targets produce nearly the same speed distribution.

6 Experiments

We report the experiments in the order we ran them, because the later ones exist to explain the earlier ones.

Constrained runs train for 5M environment steps at three seeds unless stated. The mechanism experiments in Section 6.4 train for 2.5M steps. Their control and team-mean arms have five seeds, the two PID arms have three, and the remaining two variants have two. The unconstrained baseline is a single run at 10M steps on the identical task configuration, and it did not receive a hyperparameter search of its own; it inherits the settings the constrained runs use, minus the constraint machinery. Section 6.1 states the sample-efficiency comparison with that limitation attached.

6.1 One constraint transfers

The constraint tracks its target across most of the sweep (Table 2, Figure 2). Under the probe, a linear fit to the target means gives a slope of 0.93, with a mean absolute error of 3.0 points on target means and 4.4 points per run. Training logs give a slope of 0.89 and errors of 1.5 and 2.4 points.

The probe shows two weak spots the logs hide. The two lowest targets are not separated: 0.45 and 0.55 give probe means of 0.478 and 0.496, and the speed distributions in Figure 2 nearly overlap. The seeds at 0.45 are also spread widely, from 0.368 to 0.577. At the top of the range tracking saturates, since $\tilde{d} = 0.85$ sits close to the unconstrained rate of 0.883 and the constraint has little to do.

On sample efficiency we can say less than we first thought. Five seeds of the $\tilde{d} = 0.75$ configuration trained to 2.5M steps (the control arm of Section 6.4) reach a mean tail success of 0.690, ranging from 0.622 to 0.805, against 0.657 for the single unconstrained run at 10M, both on training logs. The gap is smaller than the spread within the constrained arm. The same configuration trained to 5M reaches 0.623, below the unconstrained run rather than above it, and Section 6.3 shows why comparisons across budgets are unreliable here. Our reading is that a constrained policy at 2.5M is competitive with an unconstrained one at 10M, and nothing stronger.

Success also varies with the target, peaking at 0.55 and falling at 0.85, but the probe's seed spread reaches 0.083 at 0.45 and 0.074 at 0.75, comparable to the gaps between adjacent targets. Any claim about how a target rate affects task performance in this environment needs more than three seeds to support it.

6.2 A constraint that never binds

We had observed crowding rise under the slow constraint and added C_{crowd} as a second constraint to suppress it. Across nine runs covering three thresholds, the achieved crowding rate ended near zero regardless of which target we requested. Requesting 0.05, 0.10 and 0.15 all produced final rates between 0.011 and 0.044. The dial behaved as a switch.

To separate an interaction between the two constraints from a property of the crowding constraint itself, we ran the crowding constraint alone with the slow constraint disabled. Figure 3 shows the result and explains it. The measured crowding rate at initialization is between 0.000 and 0.012, below every threshold we swept. The constraint is an upper bound and it is satisfied before training begins. In the six runs at $\tilde{d} = 0.10$ and $\tilde{d} = 0.15$ the multiplier peaks at its initialization value and decays monotonically, never responding to a violation because none occurs. Only at $\tilde{d} = 0.05$ does the rate cross the threshold, transiently, between 1M and 2M steps, and there the multiplier does activate.

An assumption stated in passing. Roy et al. are explicit that the semantics of their thresholds hold assuming that the constraint is binding and that a feasible policy is found [8, Sec. 4.1]. Our nine runs show what happens when the first half of that assumption fails.

An inactive inequality constraint carries a zero multiplier by construction, and we do not present that as a finding about the optimizer. The finding is about the workflow the framework exists to support. A practitioner observes an unwanted behavior, picks a target rate below what they measured, and adds a constraint. That workflow fails silently here, because the behavior they measured is not the behavior the constraint sees during training. A one-sided constraint can only pull a rate toward its target from the violating side. When the policy sits on the satisfied side, the specification has no effect and produces no error, no warning and no visible difference from a run where the constraint was never added.

Bounds and rates. Section 2 draws a line between a safety budget, where undershooting is fine, and a specification, where it is a miss. The machinery we use cannot see that line, because both are written as upper bounds. If a designer means "crowding on about ten percent of ticks" rather than "at most ten percent", the constraint should say so. The direct way is to pair each upper-bound multiplier with a lower-bound multiplier on the same indicator, so that the policy is held inside a band $[\tilde{d}_k - \epsilon, \tilde{d}_k + \epsilon]$ and some multiplier is active whichever side the policy sits on. The update is the same as for the upper bound with the sign of the violation flipped, which is how the success floor already works. We did not run this variant, and Section 8 lists it as the experiment this section most needs. Whether a designer wants a band at all is a separate question. For crowding, a band would push a policy that crowds too little toward crowding more, which may not be what anyone intended. That is the point of writing it down: the one-sided version hides the choice.

Table 3: Crowding rate under the frozen-policy probe, mean over 200 episodes per policy, with bootstrap 95% intervals. All constrained policies are slow-only at $\tilde{d}_{\text{slow}} = 0.75$.

Policy	Crowding	95% interval
Unconstrained, 10M	0.028	0.014–0.045
Slow-only, 2.5M (original)	0.211	0.182–0.243
Slow-only, 2.5M, seed 1	0.131	0.107–0.159
Slow-only, 2.5M, seed 2	0.047	0.032–0.063
Slow-only, 5M, seed 1	0.017	0.006–0.031
Slow-only, 5M, seed 2	0.029	0.018–0.043
Slow-only, 5M, seed 3	0.102	0.082–0.124

Table 4: Tail-mean success by arm, training logs. The 2.5M two-constraint baseline was our original comparator and sits 11 points above the same configuration trained to 5M.

Arm	n	Success
Two-constraint, 2.5M	3	0.738
Two-constraint, 5M	3	0.623
Crowd only, 5M	9	0.635
Slow + crowd, 5M	9	0.686

Which measurement saw the side effect. The crowding that prompted the second constraint was measured with the frozen-policy probe on a slow-constrained run trained to 2.5M steps, where it reached 0.211 against an unconstrained 0.028. The training logs of the slow-constrained runs at 5M end between 0.014 and 0.079. We have since run the probe on those 5M checkpoints. Table 3 gives per-episode means.

The two instruments mostly agree, so the earlier gap was not an artifact of measurement. At 2.5M, crowding is elevated in two of the three slow-only policies we probed. At 5M it is back at the unconstrained level in two seeds and still about 3.5 times that level in the third. The side effect is real, it depends on the seed, and for most seeds it fades with a longer budget. A second probe run on the original 2.5M policy gives 0.231, inside the first run's interval. This strengthens the point above. The crowd-constrained runs trained to 5M, the budget at which most seeds had stopped crowding, and the behavior that motivated the constraint was largely gone by the time the constraint could have acted on it.

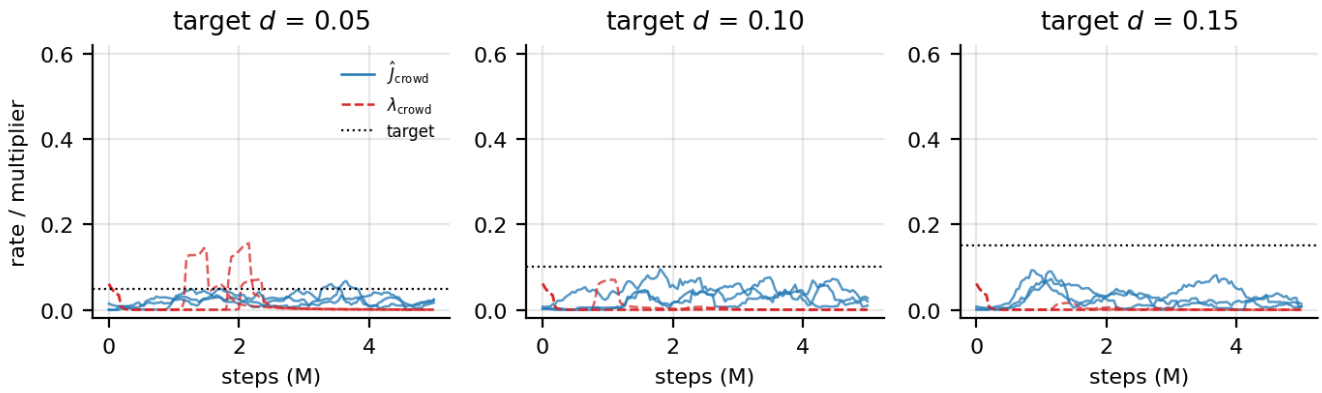


Figure 3: The crowding constraint alone, three thresholds at three seeds each, with the slow constraint disabled. The measured rate (blue) starts below the requested target (dotted) and stays there. The multiplier (red, dashed) decays from its initialization without responding to a violation, except in the $\tilde{d} = 0.05$ panel where the rate crosses the threshold between 1M and 2M steps.

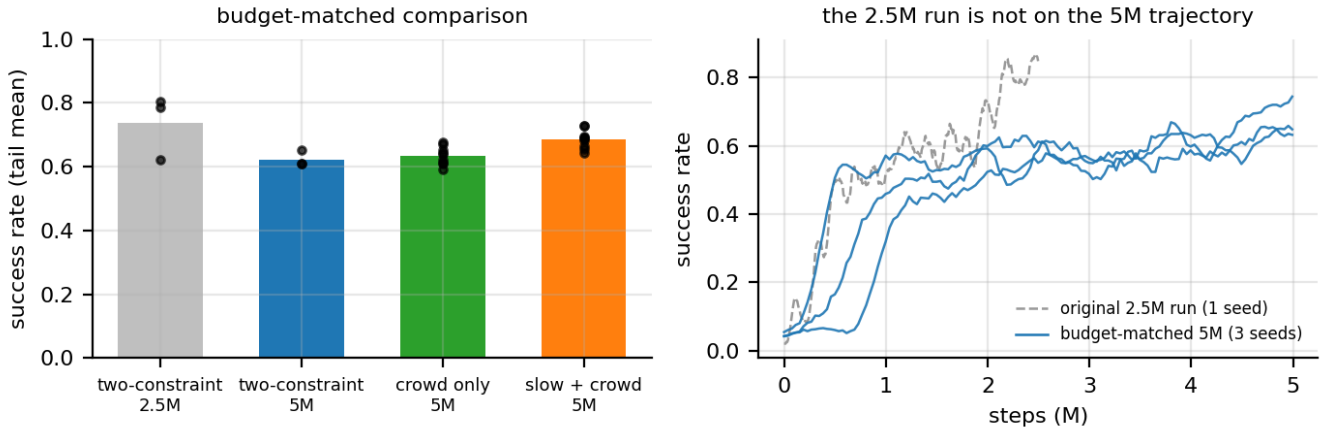


Figure 4: Left: tail-mean success by arm, individual runs overlaid. At a matched 5M budget the stacked arm sits above the two-constraint baseline rather than below it. Right: the original 2.5M run against the three budget-matched 5M runs of the same configuration. The shorter run is not an early point on the longer trajectory.

6.3 A cost that was not there

Our first comparison showed that adding the crowding constraint cost between 13 and 17 points of task success. That comparison was between crowd-constrained runs trained to 5M steps and a two-constraint baseline trained to 2.5M. We ran the baseline again at a matched budget with three seeds.

At 5M the two-constraint baseline reaches 0.623 and the stacked runs reach 0.686 (Table 4, Figure 4). The stacked arm sits above the baseline. The cost came from the budget mismatch.

Two mechanisms account for it, and both are worth naming because the original design was a natural one. First, seed variance at 2.5M is large. The five control seeds of Section 6.4 use the identical configuration and tail between 0.622 and 0.805, a spread of 18 points, and our original single-seed baseline sat near the top of that range. Second, the learning rate anneals linearly to zero over the configured budget, so at step 2.4M the 2.5M run is at 1.9×10^{-5} while the 5M run is at 1.6×10^{-4}. The shorter run has effectively converged at the point where the comparison was drawn. The two budgets are not comparable at any shared step, and the right panel of Figure 4 shows that the 2.5M run is not an early segment of the 5M trajectory.

6.4 Mechanism experiments

We run three variants at 2.5M steps, all at $\tilde{d}_{\text{slow}} = 0.75$, against a control using the full framework. Two of the three are replications of results in the original work, carried into our regime. The third, a team-mean indicator, has no single-agent counterpart, and we follow it up with a preregistered experiment that swaps the multiplier update rule. Five seeds, or three, or two in the replications, cannot establish a rate of failure. We report these as observations rather than measured frequencies.

Table 5: Mechanism experiments, 2.5M steps, $\tilde{d}_{\text{slow}} = 0.75$, training logs. Five-seed arms show the mean and, below it, the range; two-seed arms show both seeds. $\tilde{\lambda}_0$ is the weight the task reward actually carries; without a bootstrap it is λ_0. Handover is the step at which it passes from λ_{K+1} to λ_0. λ_0 alone is 0.001 to 0.007 in the team-mean arm.

Arm	n	$\hat{f}_{\text{slow}}$	$\tilde{\lambda}_0$	Handover	Success
Control	5	0.758	0.968	5/5	0.690
		0.749–0.778	0.961–0.976	1.3–2.0M	0.622–0.805
No boot.	2	0.722 / 0.702	0.983 / 0.990	n/a	0.629 / 0.621
Unnorm. λ	2	0.770 / 0.746	n/a	n/a	0.632 / 0.532
Team-mean	5	0.853	0.744	0/5	0.487
		0.626–0.996	0.654–0.856		0.369–0.562

A team-mean indicator breaks tracking (new). Replacing the per-agent slow indicator with a team-mean indicator at the same threshold fails in all five seeds. Tail rates are 0.626, 0.840, 0.822, 0.996 and 0.984 against a requested 0.75, and the probe gives 0.41, 0.98, 0.90, 0.97 and 0.99. One seed ends below target. The other four end above it, and two of those are pinned near one with a tail standard deviation under 0.02. Averaged over the whole run rather than the tail, every seed sits between 0.82 and 0.96. The rate is not satisfied on average with noise around it. In most seeds the constraint is violated for most of training. Tail success falls to between 0.369 and 0.562 against the control's 0.622 to 0.805, and mean episode length falls from between 81 and 108 ticks to between 54 and 78.

The multiplier traces show how. Early in training, in both arms, the success constraint is violated and λ_{K+1} takes nearly all of the simplex, above 0.94 by 150k steps. In the control, success crosses its floor between 1.27M and 1.97M steps, λ_{K+1} decays, and the weight passes to λ_0. In the team-mean arm that handover never happens. The slow rate sits near one through this phase, the indicator's starting point from Section 4.6. When success rises and λ_{K+1} starts to release its weight, λ_{slow} takes it instead of λ_0, reaching 0.92 to 0.996 across seeds (Figure 5, left). The slow rate drops as low as 0.2, success falls, λ_{K+1} climbs back, and the cycle repeats. We do not log the logits for these runs, so this is our reading of the λ traces rather than a measured windup.

It is tempting to read this as a collapse of the task-reward weight, since λ_0 sits between 0.001 and 0.007. That reading is wrong: the bootstrap substitutes λ_{K+1} for it, so outside the swings the task reward carries $\tilde{\lambda}_0$ between 0.65 and 0.86 (Table 5). The task is never dropped from the objective. What fails is the handover.

Unit or update rule? The account above leaves two explanations open. The counting unit could be the cause, because it puts the indicator at saturation and far from its target. Or the update rule could be the cause, because Adam climbs at a fixed rate while a constraint is violated, cannot tell a violation of 0.25 from one of 0.01, and overshoots once the policy crosses the threshold. To separate them we ran a 2×2 over {Adam, PID} × {per-agent, team-mean}. The two Adam cells are the control and team-mean arms above. The PID cells replace the logit update with a positional PID controller on the violation in the style of Stooke et al. [11], with gains fixed before any run so that under a persistent violation of 0.25 the integral term moves the logit at the same rate Adam does ($k_I = 0.12$, $k_P = 0.4$, $k_D = 1.0$, integral clamped to $[-25, 50]$). The softmax, the bootstrap, advantage standardization and the policy update are shared code across all four cells. The gains were not tuned.

Before running the PID cells we wrote down what would count as each answer. The update rule is responsible if, in at least two of three PID team-mean seeds, all four of the following hold: the tail rate is within 0.05 of 0.75; no logged λ_{slow} exceeds 0.5; the bootstrap hands over; and tail success lies inside the Adam control's seed range. The counting unit is responsible if λ_{slow} still swings above 0.5 with no handover. Anything else is mixed. Criteria were scored on training-log tail means.

Table 6 and Figure 5 give the result. On the preregistered measure, two of three PID team-mean seeds pass all four criteria, which makes the verdict "update rule". All three track to within 0.005 on the log tail, and all three hand over, between 0.88M and 1.27M steps. The third seed fails two criteria: λ_{slow} reaches 0.546 once, and its tail success of 0.570 falls below the control range. Under PID the training rate still swings widely, with a tail standard deviation between 0.13 and 0.18. It is satisfied on average with variance around the target, which is what we would have hoped the Adam arm did and what it did not do.

Three results qualify the verdict, and we think they matter more than the verdict itself.

First, the probe disagrees with the logs. Section 5 prefers the probe. Under the probe, the PID team-mean seeds give 0.717, 0.779 and 0.592, and the third seed misses by 0.16. Scored on the probe, only one seed passes all four criteria, which our own rule calls mixed. We preregistered the log measure and report its verdict, but we would not rest a claim on a result that changes with the instrument.

Second, PID breaks the per-agent control. The Adam control tracks to within 0.03 in all five seeds. The PID control ends at 0.444, 0.714 and 0.597, so two of three seeds miss by more than 0.05. All three are on the satisfied side of an upper bound, which by Section 6.2 means nothing pulls them back toward 0.75. The seed furthest below target has the highest success, 0.877. This is the one-sided problem again, arriving late in training instead of at initialization. Our preregistration did not include a criterion for this cell, and we did not anticipate it.

Third, the gains were set once and never tuned, so we cannot say whether a different PID setting would track in both units.

Taken together, neither the counting unit nor the update rule explains the failure alone. Adam tracks the per-agent indicator and fails on the team-mean one; PID tracks the team-mean indicator, on logs, and loses the per-agent one. Our reading is that the team-mean unit puts the indicator at saturation, far above its target at the moment the success constraint releases its weight, and that Adam's fixed-rate logit climb turns that into a switch between λ_{K+1} and λ_{slow}. PID damps the switch but overshoots below the target when the indicator is not saturated, and an upper bound cannot correct an overshoot in that direction.

Unnormalized multipliers (replication). Roy et al. motivate Equation (5) with a scenario in which an unsatisfiable constraint drives its multiplier without bound until the critic loss diverges and performance collapses [8, Fig. 3]. Replacing the softmax with independent non-negative multipliers reproduces this in one seed of

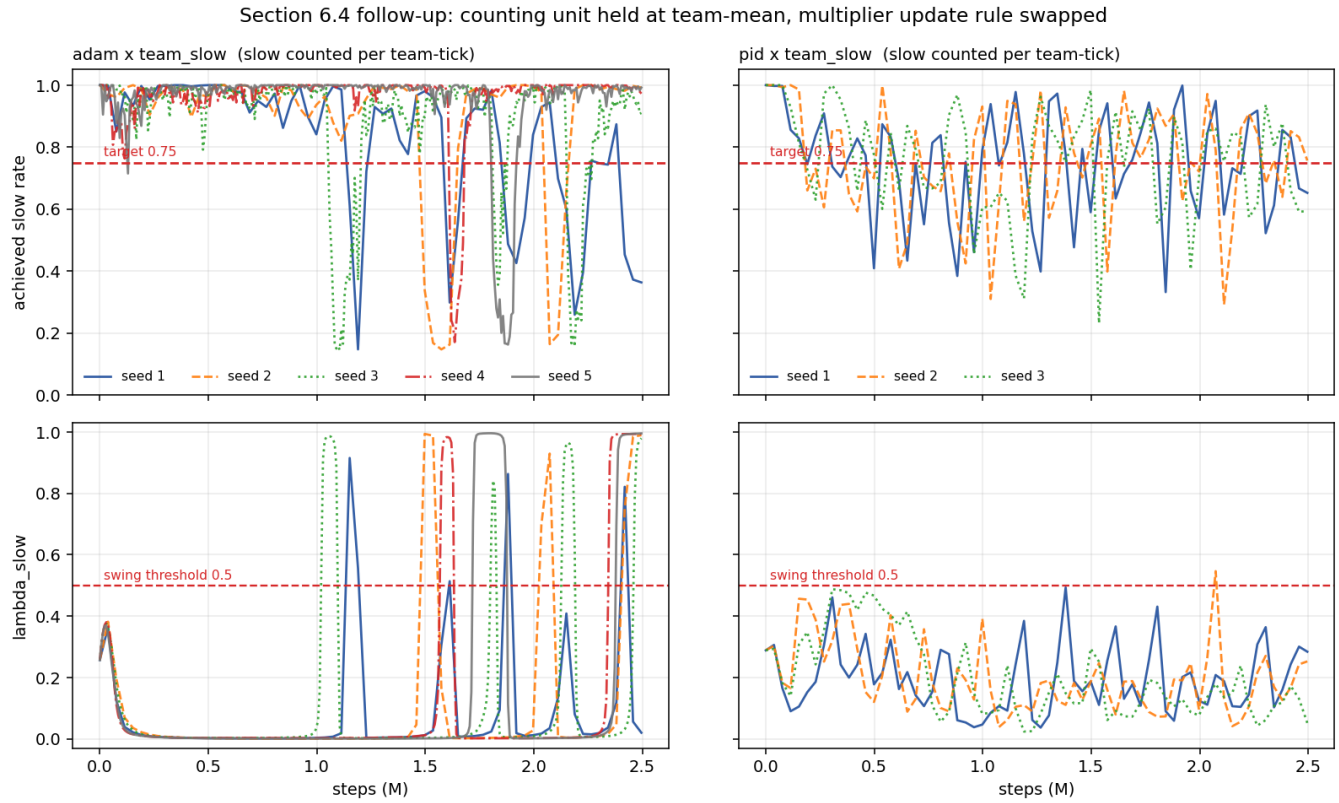


Figure 5: The team-mean indicator under two multiplier update rules, training logs. Left: Adam, five seeds. Right: PID, three seeds. Top: achieved team-mean slow rate, target 0.75 dashed. Bottom: λ_{slow}, with the preregistered swing threshold of 0.5 dashed. Under Adam the rate sits near one and λ_{slow} alternates between near zero and near one. Under PID the rate oscillates around the target and λ_{slow} stays below 0.55.

Table 6: The 2×2 over multiplier update rule and counting unit, 2.5M steps, $\tilde{d}_{\text{slow}} = 0.75$. Each cell gives the mean with the range across seeds in parentheses. Log and probe rates count whatever unit the arm's indicator counts. Max λ_{slow} is the largest logged value over the whole run.

Update	Unit	n	Log $\hat{f}_{\text{slow}}$	Probe $\hat{f}_{\text{slow}}$	Max λ_{slow}	Handover	Success
Adam	per-agent	5	0.758 (0.749–0.778)	0.754 (0.646–0.855)	0.26–0.35	5 of 5	0.690 (0.622–0.805)
Adam	team-mean	5	0.853 (0.626–0.996)	0.851 (0.411–0.988)	0.92–1.00	0 of 5	0.487 (0.369–0.562)
PID	per-agent	3	0.585 (0.444–0.714)	0.581 (0.461–0.698)	0.28–0.29	3 of 3	0.697 (0.578–0.877)
PID	team-mean	3	0.751 (0.748–0.754)	0.696 (0.592–0.779)	0.49–0.55	3 of 3	0.632 (0.570–0.688)

two: λ_{K+1} reaches 30.2 and λ_{slow} reaches 1.40, success falls to 0.532, and episode length collapses to 60 ticks. The other seed is indistinguishable from control. Their demonstration used a deliberately infeasible constraint; ours arises without one, which suggests the instability can appear in ordinary training, though we cannot say how often.

Removing the bootstrap has no effect here (replication, negative). Roy et al. show that without the bootstrap constraint, an agent under five simultaneous constraints settles on a trivial constraint-satisfying behavior and never solves the main task [8, Fig. 5b]. We do not reproduce this. Achieved rates are 0.722 and 0.702, inside or just below the control range, success is 0.629 and 0.621, at the bottom of the control range, and episode lengths are comparable.

The likely reason is consistent with their account rather than against it. The bootstrap exists because $\lambda_0 = 1 - \sum_k \lambda_k$ is squeezed toward zero as constraints accumulate. Our variant runs one behavioral constraint plus success, and λ_0 sits at 0.98 throughout, so there is no squeeze to relieve. We read this as evidence that the mechanism's necessity scales with constraint count and that it is inert at $K = 1$, not as a failure of the mechanism.

7 Discussion

A target rate specifies a behavior only when the multiplier attached to it has something to respond to, and our two failures are two ways that goes wrong.

The first is the binding assumption Roy et al. state in passing [8, Sec. 4.1]. An upper bound only acts while the policy is above the threshold. Our crowding constraint was below its threshold from initialization onward and did nothing. The PID control was above its threshold at the start and then overshot below it, after which nothing brought it back. Both are the same gap between what the designer wrote and what the machinery enforces. A designer who writes 0.10 usually means a rate, and an upper bound enforces a ceiling. Pairing each upper-bound multiplier with a lower-bound one closes that gap, and it is the variant we would run next.

The second is about how weight moves through the simplex. The multipliers share one unit of weight, and the bootstrap means the task's share passes through the success constraint first. When the behavioral indicator is saturated at the moment success releases its weight, as the team-mean indicator is, an Adam update hands that weight to the behavioral constraint instead of the task, and the two constraints trade it back and forth. The counting unit matters here, but less directly than we first claimed. It changes what a threshold of 0.75 means, and it decides where the indicator starts. Whether that start breaks training depends on the update rule.

Neither failure is visible from the target rate. A practitioner reading $\tilde{d}_{\text{crowd}} = 0.10$ and $\tilde{d}_{\text{slow}} = 0.75$ has no way to tell that the first is inert and the second is not, and neither run reports an error. Three checks catch both, and each costs one evaluation pass of the untrained policy. For an upper bound, confirm the requested target sits below the indicator's starting rate. Confirm that the starting rate is not saturated near zero or one, where the indicator has little room to move and the multiplier dynamics are least predictable. And decide whether the number written down is a ceiling or a rate, and if it is a rate, bound it from both sides.

Direct behavior specification does transfer to a harder regime than the one it was built for. What does not transfer automatically is the assumption that writing down a rate is the same as specifying a behavior.

8 Limitations

Seeds on the mechanism experiments. The control and team-mean arms have five seeds, the PID arms three, and the no-bootstrap and unnormalized-multiplier variants two. The team-mean failure is consistent across all five seeds. The unnormalized-multiplier result splits across its two seeds, and we cannot report a failure rate for it.

The PID verdict depends on the instrument. Our preregistered criteria were scored on training logs and give "update rule". Scored on the frozen-policy probe, which we otherwise prefer, the same criteria give "mixed". We report both. The preregistration also had no criterion for the PID per-agent cell, which turned out to be the most informative of the four.

One set of PID gains. We set the PID gains once, to match Adam's rate of climb under a persistent violation, and did not tune them. The per-agent failure under PID may be a property of these gains rather than of PID.

The two-sided band is untested. Sections 6.2 and 7 argue that a target rate should be bounded from both sides. We have not run that variant, on crowding or on the PID per-agent cell, and we do not know whether a band tracks where the one-sided bound fails.

Crowding at 5M.. The 5M probe shows crowding at the unconstrained level in two of three seeds and elevated in the third. Three seeds cannot tell us how often the side effect persists at the longer budget.

Seed variance in the slow sweep. Under the probe the seed spread in success reaches 0.083 at $\tilde{d} = 0.45$, comparable to the gaps between adjacent targets, and the probe does not separate the two lowest targets. Conclusions about how the requested rate trades against task performance are correspondingly weak.

At most two constraints. Roy et al. handle up to five simultaneous constraints, and their bootstrap mechanism exists for that setting. Our experiments use at most two, so we say nothing about how either failure behaves as the constraint set grows. The simplex competition in Section 6.4 would plausibly get worse with more constraints, but we have not tested it.

Single task and single zone. Every result comes from one scenario with one target zone. We do not know how much of the tracking behavior depends on that geometry.

9 Conclusion

Direct behavior specification via constrained reinforcement learning transfers to a shared-policy multi-agent setting trained on-policy against a reactive opponent, for an upper bound the policy actually violates. It does nothing for an upper bound the policy already satisfies, and nothing pulls a policy back once it overshoots below one. Counting the indicator over the team breaks tracking under Adam, and a preregistered comparison shows that the update rule and the counting unit cause that failure together. These conditions can be checked before training, and none is visible from the specification. We also report a 13 to 17 point effect that disappeared under a budget-matched baseline, because the comparison that produced it is one others are likely to draw.

Acknowledgments

References

[1] Joshua Achiam, David Held, Aviv Tamar, and Pieter Abbeel. 2017. Constrained Policy Optimization. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*. 22–31.

[2] Eitan Altman. 1999. *Constrained Markov Decision Processes*. Vol. 7. CRC Press.

[3] Dario Amodei, Chris Olah, Jacob Steinhardt, Paul Christiano, John Schulman, and Dan Mané. 2016. Concrete Problems in AI Safety. *arXiv preprint arXiv:1606.06565* (2016).

[4] Shangding Gu, Jakub Grudzien Kuba, Yuanpei Chen, Yali Du, Long Yang, Alois Knoll, and Yaodong Yang. 2023. Safe Multi-Agent Reinforcement Learning for Multi-Robot Control. *Artificial Intelligence* 319 (2023), 103905.

[5] Victoria Krakovna, Laurent Orseau, Ramana Kumar, Miljan Martic, and Shane Legg. 2018. Penalizing Side Effects using Stepwise Relative Reachability. *arXiv preprint arXiv:1806.01186* (2018).

[6] Karol Kurach, Anton Raichuk, Piotr Stańczyk, Michał Zając, Olivier Bachem, Lasse Espeholt, Carlos Riquelme, Damien Vincent, Marcin Michalski, Olivier Bousquet, and Sylvain Gelly. 2020. Google Research Football: A Novel Reinforcement Learning Environment. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 34. 4501–4510.

[7] Chenyi Liu, Nan Geng, Vaneet Aggarwal, Tian Lan, Yuan Yang, and Mingwei Xu. 2021. CMIX: Deep Multi-Agent Reinforcement Learning with Peak and Average

Constraints. In *Machine Learning and Knowledge Discovery in Databases (ECML PKDD)*.

[8] Julien Roy, Roger Girgis, Joshua Romoff, Pierre-Luc Bacon, and Christopher Pal. 2022. Direct Behavior Specification via Constrained Reinforcement Learning. In *Proceedings of the 39th International Conference on Machine Learning (ICML) (PMLR, Vol. 162)*.

[9] William Spearman. 2018. Beyond Expected Goals. In *MIT Sloan Sports Analytics Conference*.

[10] William Spearman, Austin Basye, Greg Dick, Ryan Hotovy, and Paul Pop. 2017. Physics-Based Modeling of Pass Probabilities in Soccer. In *MIT Sloan Sports Analytics Conference*.

[11] Adam Stooke, Joshua Achiam, and Pieter Abbeel. 2020. Responsive Safety in Reinforcement Learning by PID Lagrangian Methods. In *Proceedings of the 37th International Conference on Machine Learning (ICML)*. 9133–9143.

[12] Chen Tessler, Daniel J. Mankowitz, and Shie Mannor. 2019. Reward Constrained Policy Optimization. *International Conference on Learning Representations (ICLR)* (2019).

[13] Alexander Matt Turner, Dylan Hadfield-Menell, and Prasad Tadepalli. 2020. Conservative Agency via Attainable Utility Preservation. In *Proceedings of the AAAI/ACM Conference on AI, Ethics, and Society*.

[14] Zhe Wang, Petar Veličković, Daniel Hennes, Nenad Tomašev, Laurel Prince, Michael Kaisers, Yoram Bachrach, Romuald Elie, Li Kevin Wenliang, Federico Piccinini, William Spearman, Ian Graham, Jerome Connor, Yi Yang, Adrià Recasens, Mina Khan, Nathalie Beauguerlange, Pablo Sprechmann, Pol Moreno, Nicolas Heess, Michael Bowling, Demis Hassabis, and Karl Tuyls. 2024. TacticAI: An AI Assistant for Football Tactics. *Nature Communications* 15, 1 (2024), 1906.